Kapruka Gift Concierge

Technical Architecture Document

A full-screen, MCP-connected AI shopping agent for Kapruka.com

---

Prepared for: Kapruka Agent Challenge 2026 submission

Author: Erica Jayasundera

(BSc Hons in Software Engineering, MSc in Information Security , Phd Candidate)

Repository: github.com/ericajaya/kapruka_gift_concierge

Document date: 1 July 2026

1. Executive Summary

Kapu is a full-screen, chat-native AI shopping agent built for the Kapruka Agent Challenge. It turns the free, public Kapruka MCP server into a polished, end-to-end gifting experience: a customer can discover products, receive curated recommendations, check delivery feasibility, build a multi-item cart, and complete a real guest-checkout order — all inside a single conversational interface, with no account and no database.

The system is deliberately thin. It holds no application state of its own: catalog data, delivery quotes, and orders are fetched live from Kapruka on every turn, and conversation/cart state lives entirely in the browser. The only persistent piece of server-side logic is a single Next.js API route that mediates between the browser, the Claude Messages API, and the Kapruka MCP server.

Core design principle: nothing shown to the customer is invented. Every product, price, delivery quote, and order confirmation is grounded in a live tool call — the LLM is instructed never to fabricate catalog data, and the UI only renders what the MCP server actually returned.

2. System Architecture

The architecture has four tiers: the browser client, a thin Next.js server acting purely as an SSE relay and prompt builder, the Claude Messages API (which owns the tool-calling loop via Anthropic's MCP connector), and the third-party Kapruka MCP server. Anthropic's infrastructure — not application code — performs the MCP handshake and tool execution loop; the Next.js route only supplies configuration and re-streams the result.

Figure 1 — Four-tier architecture: browser client, Next.js relay, Claude Messages API (MCP connector), Kapruka MCP server.

2.1 Why an MCP connector instead of a custom tool loop

Rather than implementing tool-call orchestration by hand (dispatch → execute → append result → re-call the model), the API route delegates the entire loop to Anthropic's MCP connector beta. The request simply declares the remote server and an mcp_toolset; Claude negotiates with mcp.kapruka.com directly and the response stream already contains interleaved mcp_tool_use and mcp_tool_result content blocks. This removes an entire class of orchestration bugs and keeps the Next.js route stateless and short.

3. Request Lifecycle

Every customer turn follows the same seven-step path, shown below. Steps 3–4 may repeat multiple times within a single turn if Claude decides to call more than one Kapruka tool (for example, a delivery check followed by an order creation).

Figure 2 — Request lifecycle for a single chat turn, from keystroke to rendered result.

3.1 Streaming protocol

The API route (app/api/chat/route.js) opens a raw fetch stream to the Anthropic Messages API with stream: true and manually parses the resulting Server-Sent Events, rather than using the SDK's higher-level stream helper — this was a deliberate choice to keep behaviour uniform across Node runtimes. It re-emits a simplified event set to the browser over its own SSE response:

```
chunk  — a text token to append to the visible reply
tool   — a tool name, the first time it is invoked in this turn
done   — the final message_stop event, carrying parsed
         products / orders / delivery / tracking / errors
error  — a request or stream-level failure
```

Content blocks are accumulated server-side in an indexed map as content_block_start / _delta / _stop events arrive. Tool-call JSON input and tool-result text are both assembled incrementally, then finalised on content_block_stop. When message_stop fires, the full ordered list of content blocks is handed to the parsing layer (lib/parseMcp.js) before the done event closes the stream.

4. Backend: API Route

The route accepts a single POST with the running message history plus lightweight session context (cart contents, previously shown products, and the active UI language), and returns a streamed response. Its responsibilities are narrow and explicit:

- Validate the request body and required environment variables (fails fast with a structured JSON error if ANTHROPIC_API_KEY is missing).
- Build a system prompt via lib/persona.js, injecting the current cart and previously-seen products so Claude has continuity without a database.
- Call the Anthropic Messages API with the Kapruka MCP server attached as an mcp_servers entry and tools: [{ type: "mcp_toolset" }].
- Relay text deltas and tool-call announcements to the browser in real time.
- On stream completion, hand the accumulated content blocks to the MCP result parser and forward the structured result.

Key configuration constants

```
MCP_BETA        = "mcp-client-2025-11-20"   // Anthropic beta header
MCP_SERVER_NAME = "kapruka"
model           = process.env.ANTHROPIC_MODEL || "claude-sonnet-4-6"
mcp_servers     = [{ type: "url", url: KAPRUKA_MCP_URL, name: "kapruka" }]
tools           = [{ type: "mcp_toolset", mcp_server_name: "kapruka" }]
```

The route is stateless per request — runtime: "nodejs" with maxDuration: 60 — and holds no session data between calls; every piece of context the model needs is re-sent by the client on each turn.

5. Kapruka MCP Integration

The Kapruka MCP server (mcp.kapruka.com/mcp) is a free, public, streamable-HTTP endpoint requiring no API key, rate-limited to 60 requests/minute and 30 order-creation calls/hour per client IP. It exposes seven tools, all of which the agent is instructed to use rather than improvise:

Tool	Purpose	Notes
kapruka_search_products	Keyword search with category, price range, stock and sort filters	Pagination capped at 3 pages
kapruka_get_product	Full detail for a single product by ID	Used by the Lightbox detail view
kapruka_list_categories	Top-level category names and browse URLs	Feeds category-scoped searches
kapruka_list_delivery_cities	Fuzzy city lookup by canonical or vernacular name	Up to 50 matches per query
kapruka_check_delivery	Delivery feasibility, flat LKR rate, and perishable warnings	Drives the inline date picker
kapruka_create_order	Creates a guest-checkout order and returns a pay link	60-minute price lock; 30/hr cap
kapruka_track_order	Status and delivery progress for an existing order	Keyed by customer-supplied order number

Table 1 — Kapruka MCP tool surface consumed by Kapu.

6. Data Normalisation Layer

The Kapruka MCP server may return either structured JSON or markdown-formatted text inside its mcp_tool_result content blocks. lib/parseMcp.js insulates the rest of the application from this ambiguity with a JSON-first, regex-fallback strategy:

- extractToolData() walks the finished content-block array, pairs each mcp_tool_result with its originating mcp_tool_use by tool_use_id, and routes the result text to a type-specific parser based on the tool name.
- parseProducts() first attempts JSON.parse (after stripping ```json fences); if that fails, it splits the text into candidate blocks on numbered/bulleted/heading boundaries and extracts name, price, product ID, stock status, URL, and image via targeted regular expressions.
- parseOrder() extracts order number, pay-URL, total, currency, and the price-lock window, with the same JSON-then-regex fallback.
- parseDelivery() and parseTracking() preserve the raw text alongside a best-effort structured object, since these are rendered largely as prose in the UI.
- Every normalised product passes through normaliseProduct(), which reconciles multiple possible field names (id/product_id/sku, image/image_url/thumbnail, etc.) into one consistent shape for the React layer.

Engineering note: this file was written without live access to the MCP server from the build environment, so its regex fallbacks are a best-effort defensive layer. The README explicitly flags it as the first place to adjust if a live field (image, price, stock) doesn't render correctly once connected to the real endpoint.

7. LLM Persona and Prompt Engineering

lib/persona.js defines Kapu's voice and operating rules as a static base prompt, composed at request time with dynamic session context. The persona is deliberately constrained: it must ground every claim in a tool result, curate rather than dump raw search results, and never fabricate a product, price, or availability.

7.1 Behavioural rules encoded in the system prompt

- Always call the search tool for any discovery request; never invent a product.
- Curate 2–4 strong recommendations per query rather than listing everything returned.
- Only call the order tool once the customer has confirmed the cart and delivery details — or, when the UI's Checkout Wizard has already assembled a payload, treat an embedded ```json ORDER_REQUEST block in the user message as the authoritative source of truth for that call.
- Proactively offer to add a gift message when the customer appears to be buying for someone else.
- Report tool failures honestly in one short sentence rather than guessing at a result.

7.2 Language handling

A single language parameter (en / si / ta / tanglish) selects a language directive appended to the system prompt. Sinhala and Tamil directives instruct Claude to reply primarily in-script while keeping product names, prices, order numbers, and URLs in English/numerals for lack of ambiguity; the Tanglish directive asks for casual English seasoned with everyday local expressions.

7.3 Session continuity without a database

Because no server-side store exists, buildSystemPrompt() re-injects the customer's current cart and up to the last 20 previously-shown products (by ID and name) into every system prompt, giving Claude short-term memory of what has already been discussed without any persistence layer.

8. Client-Side State and UI Architecture

app/page.js is the single source of truth for UI state: messages, input, language, theme, cart, knownProducts, and various modal/loading flags. It has no external state manager — all coordination is done with React's built-in useState and prop drilling into a focused set of presentational components.

Component	Responsibility
Sidebar.jsx	Logo, new-conversation action, starter prompts
Hero.jsx	Empty-state greeting, quick-prompt cards, occasion tags, welcome-back message
Header.jsx	Brand bar, language toggle (EN / SI / TA / Tanglish), theme toggle, mobile menu
Composer.jsx	Message input, disabled image-upload affordance, Web Speech API voice input, send
MessageBubble.jsx	Renders a chat turn: streaming text, tool-call badges, product/order cards
ProductTag.jsx	Die-cut “gift-tag” product card and carousel; opens the Lightbox on click

Component	Responsibility
Lightbox.jsx	Full-screen product detail overlay with an add-to-cart action
BudgetChips.jsx	Price-range quick filters shown beneath a set of product results
DatePickerWidget.jsx	Inline delivery-date picker surfaced after a delivery tool call
OrderCard.jsx	Order confirmation card with pay-link and price-lock countdown
CheckoutWizard.jsx	6-step guided checkout: bundle review → recipient → address → date → gift message → review
Toast.jsx	Bottom-centre transient confirmation (e.g. “Added to cart”)

Table 2 — Frontend component responsibilities.

8.1 Checkout flow

Checkout is deliberately not a raw form POST to the MCP tool. The CheckoutWizard collects a structured payload (cart, recipient, delivery, gift message) client-side, then hands off to the existing conversational path: page.js serialises the payload as a fenced ```json ORDER_REQUEST block inside a synthetic user message, and sends it through the normal /api/chat flow. Kapu's system prompt explicitly recognises this block as authoritative and calls kapruka_create_order with that exact data, which keeps checkout inside the same audit trail as the rest of the conversation rather than opening a parallel code path.

8.2 Session memory

lib/sessionMemory.js wraps localStorage with SSR- and quota-safe try/catch guards. It persists the last recipient name and city (opportunistically extracted from user messages with lightweight regex) and the timestamp of the last completed order, purely to power a “Welcome back — shopping for X again?” hero message on return visits. No PII leaves the browser and no server-side record is kept.

9. Technology Stack

Layer	Technology	Notes
Framework	Next.js 16 (App Router)	Single API route + client-rendered page
UI runtime	React 19	useState-based state, no external state library
Styling	Hand-written CSS (globals.css)	Design-token based; light theme default, dark via [data-theme]
LLM	Claude (claude-sonnet-4-6)	Configurable via ANTHROPIC_MODEL
Tool connectivity	Anthropic MCP connector (beta)	anthropic-beta: mcp-client-2025-11-20
Catalog / commerce backend	Kapruka MCP server	Third-party, free, public, no auth
Persistence	None (server) / localStorage (client)	No database anywhere in the stack

Layer	Technology	Notes
Voice input	Browser Web Speech API	Chrome/Edge; degrades to inert elsewhere

10. Environment Configuration

Variable	Required	Default	Purpose
ANTHROPIC_API_KEY	Yes	—	Authenticates calls to the Claude Messages API
KAPRUKA_MCP_URL	No	https://mcp.kapruka.com/mcp	Override only if Kapruka issues a different endpoint
ANTHROPIC_MODEL	No	claude-sonnet-4-6	Any current Claude model supporting the MCP connector beta

11. Deployment Architecture

The application is designed for zero-infrastructure deployment: any Node 18+ host running npm run build && npm run start will serve it, with Vercel as the recommended path (auto-detects Next.js, free tier is sufficient, single environment variable to set). Because there is no database and no server-side session store, the deployment is horizontally stateless — any number of instances can run behind a load balancer with no shared state required.

- Push the repository to GitHub and import it at vercel.com/new.
- Set ANTHROPIC_API_KEY (and optionally KAPRUKA_MCP_URL / ANTHROPIC_MODEL) as environment variables.
- Deploy — the resulting *.vercel.app URL (or an attached custom domain) is the live link submitted for judging.

12. Known Limitations and Engineering Notes

These are documented honestly in the project README rather than hidden, in keeping with the challenge's emphasis on shipping something real:

- The markdown-fallback branch of lib/parseMcp.js was written without live access to the Kapruka MCP server and should be re-verified against real search results before submission; field-mapping regexes are the most likely point of drift.
- The composer's image-upload icon is intentionally left visibly disabled — there is no backend support for image-based search, so it is not wired to fake functionality.

- The inline delivery-date picker's visibility is governed by a heuristic (shown after any delivery tool call, hidden once an order exists), which is reasonable for a demo but could misfire in a longer conversation with multiple delivery checks.
- Voice input depends on browser support for the Web Speech API and silently no-ops where unsupported, rather than erroring.
- All rate limiting (60 requests/minute, 30 orders/hour) is enforced by the Kapruka MCP server itself, per client IP — the application performs no additional throttling.

13. Challenge Rubric Alignment

Rubric item	How it is addressed
Full-screen chat UI (30 pts: experience & polish)	app/page.js occupies the entire viewport; no embedded widget
Visual richness (20 pts)	Gift-tag product cards with images/price/stock; order confirmation card; lightbox detail view
Personality (15 pts)	Kapu's voice defined in lib/persona.js — warm, concise, opinionated
Usefulness (15 pts)	Curated 2–4 picks per query; live delivery checks and order tracking built in
End-to-end completeness (15 pts)	Search → cart → Checkout Wizard → real kapruka_create_order call → pay link
Creativity (5 pts)	Die-cut gift-tag visual motif tied to the gifting theme
Bonus — multi-item carts	CheckoutWizard supports any number of line items with quantity steppers
Bonus — delivery-date constraints	Inline DatePickerWidget feeds a chosen date back into the conversation
Bonus — gift messaging	Optional gift-message field passed through to kapruka_create_order
Bonus — Tanglish / Sinhala	Four-way language toggle (EN / SI / TA / Tanglish) steers Kapu's reply language